

Assignment 1 Solutions

Process Abstractions, Scheduling & Performance Analysis

1. Process Abstraction & the POSIX API

1.1 State Machines Under Pressure

(a) Timer Interrupt and Context Switch

When the hardware timer fires while a process is executing:

1. CPU switches from user mode to kernel mode.
2. Current process context (PC, registers, stack pointer, flags) is saved into the PCB.
3. Kernel interrupt handler runs.
4. Running process transitions from Running $\rightarrow$ Ready.
5. Process PCB inserted into the ready queue/run queue.
6. Scheduler selects another ready process.
7. Context of selected process restored from its PCB.
8. MMU/address space metadata updated if required.
9. Selected process transitions Ready $\rightarrow$ Running.
10. CPU resumes execution of the selected process.

Relevant data structures:

- PCB (Process Control Block)
- Ready queue / run queue
- Trap frame / saved register context
- Scheduler metadata

(b)(i) Invalid Transitions

Student sequence:

Running $\rightarrow$ Terminated $\rightarrow$ Blocked $\rightarrow$ Ready $\rightarrow$ Running

Invalid transitions:

1. Running $\rightarrow$ Terminated

Invalid because `read()` on an empty socket does not terminate the process.

2. Terminated $\rightarrow$ Blocked

Impossible because a terminated process no longer executes.

(b)(ii) Correct Transition Sequence

Correct sequence:

Running $\rightarrow$ Blocked $\rightarrow$ Ready $\rightarrow$ Running

- Running $\rightarrow$ Blocked:
Process calls `read()` with no data available.
- Blocked $\rightarrow$ Ready:
Network interrupt signals data arrival.
- Ready $\rightarrow$ Running:
Scheduler dispatches process onto CPU.

(c) Zombie Processes

A zombie process retains an entry in the process table because the kernel must preserve:

- PID
- exit status
- accounting information

The zombie is removed when the parent calls:

`wait()`

or

`waitpid()`

If the parent exits first:

- the zombie is adopted by `init/systemd`,
- which later reaps it automatically.

1.2 Deep fork() Analysis

(a) Process Proliferation

For:

```
for(i=0;i<n;i++)  
    fork();
```

Total processes:

$$2^n$$

Reason: Each iteration doubles the number of existing processes.

For the given program:

- Process A = original parent
- Process B = child from Fork A
- Process C = child from Fork B

Hence total processes:

$$3$$

(b) Exact Output

Possible output:

C: g=110 x=105

B: g=10 x=5

A: g=10 x=5

Explanation:

- Process C modifies:

$$g = 110, \quad x = 105$$

- Parent and sibling processes retain independent copies of memory due to copy-on-write after `fork()`.

(c) Ordering Guarantees

Because:

- B waits for C
- A waits for B

Ordering constraint:

$$C \rightarrow B \rightarrow A$$

Impossible orderings:

- A before B
- A before C
- B before C

(d) Modified Scenario

(i) Shared Heap Object? No.

After `fork()`, parent and child have logically separate address spaces using copy-on-write.

(ii) Possible Bug Possible bug:

- use-after-free
- dangling pointer

Detection:

- Valgrind
- AddressSanitizer (ASan)

(iii) Orphan Process If Process B exits before waiting for C:

- Process C becomes orphaned.
- It is adopted by PID 1 (`init/systemd`).

Repeated occurrence may cause zombie accumulation and process table exhaustion.

2. Scheduling Algorithms & Trade-offs

2.1 Multi-Process Workload Analysis

Process	Arrival	Burst	Priority
P1	0	10	3
P2	2	4	1
P3	4	6	2
P4	6	2	4
P5	8	8	2

(a)(i) SRTF Preemptions

At $t = 2$:

$P1 \rightarrow P2$

because:

$4 < 8$

At $t = 6$:

$P3 \rightarrow P4$

because:

$2 < 6$

(a)(ii) Can P1 Finish Last?

Yes.

Under SRTF, repeated arrival of shorter jobs can continually preempt P1.

(b) Why SJF Beats FCFS

FCFS suffers convoy effect: short jobs wait behind long process P1.

SJF schedules shorter jobs earlier, reducing cumulative waiting time.

(c) Round Robin ($q = 3$ ms)

Execution order:

0-3 P1
3-6 P2
6-9 P1
9-12 P3
12-14 P2
14-16 P4
16-19 P5
19-22 P1
22-25 P3
25-28 P5
28-29 P1
29-31 P5

Turnaround Times

Process	TAT
P1	29
P2	12
P3	21
P4	10
P5	23

Average turnaround time:

$$\frac{29 + 12 + 21 + 10 + 23}{5} = 19$$

Wait Times

Process	WT
P1	19
P2	8
P3	15
P4	8
P5	15

Average wait time:

$$\frac{19 + 8 + 15 + 8 + 15}{5} = 13$$

(d) Non-preemptive SJF

Execution order:

P1: 0-10

P4: 10-12

P2: 12-16

P3: 16-22

P5: 22-30

Wait Times

$$\frac{0 + 4 + 10 + 12 + 14}{5} = 8$$

Turnaround Times

$$\frac{10 + 6 + 14 + 18 + 22}{5} = 14$$

2.2 Theoretical Bounds

(a) SJF Optimality

For:

$$b_1 \leq b_2 \leq b_3$$

SJF ordering:

$$b_1, b_2, b_3$$

Average wait time:

$$\frac{0 + b_1 + (b_1 + b_2)}{3} = \frac{2b_1 + b_2}{3}$$

Any permutation placing a larger burst earlier increases cumulative waiting.
Hence SJF minimizes average waiting time.

(b) SRTF vs RR

SRTF Short jobs respond immediately:

$$\text{Average response} \approx 0$$

RR Worst-case response:

$$(k-1)q$$

Average response:

$$\frac{q(k-1)}{2}$$

RR improves fairness while SRTF minimizes turnaround.

3. Quantitative Performance & Advanced Metrics

3.1 Metric Derivation

Process	Start	Finish
P1	0	12
P2	12	15
P3	15	22
P4	22	24
P5	24	30

(a) Wait and Turnaround Times

Process	WT	TAT
P1	0	12
P2	12	15
P3	15	22
P4	22	24
P5	24	30

(b) Makespan

$$30 - 0 = 30$$

(c) Jain's Fairness Index

Wait vector:

$$[0, 12, 15, 22, 24]$$

$$\sum x_i = 73$$

$$\sum x_i^2 = 1429$$

$$J = \frac{73^2}{5(1429)} \approx 0.746$$

(d) Comparing FCFS and SJF Fairness

SJF increases variance in wait times:

- short jobs wait less,
- long jobs wait more.

Therefore:

$$J_{SJF} < J_{FCFS}$$

3.2 Interpreting Real-World Measurements

(a) Algorithm Family

Most likely:

- SJF/SRTF-like scheduler

because:

- very low mean wait,
- extremely high tail latency.

(b) Throughput vs Tail Latency

A scheduler can minimize makespan by maximizing CPU utilization while still producing poor interactive latency.

Makespan measures throughput, not responsiveness.

Thus it is not ideal for interactive systems.

(c) Deployment Decisions

Batch Genomics Pipeline Choose:

Gamma

Reason:

- lowest makespan,
- highest throughput.

Real-Time Chat Application Choose:

Delta

Reason:

- lowest P90 wait,
- lowest P99 wait.